SaleSphere — Script

1. Purpose and Scope

The purpose of this chaos engineering script is to test how SaleSphere behaves when different failures are deliberately introduced into the application. SaleSphere is a self-healing CRM project that uses PatchPilot to monitor failures and perform configured recovery actions.

The main objective is to demonstrate that the system can detect a problem, start the appropriate recovery process, and verify whether normal operation has been restored.

The experiments covered in this script are:

- **Bad Software Release:** Introduce a faulty software version and test automatic rollback.
- **MongoDB Disconnection:** Interrupt database connectivity and test offline data handling and synchronization.
- **Email Failure:** Simulate an email delivery failure and test the configured retry mechanism.
- **API Failure:** Introduce a controlled API error and observe failure detection and recovery behavior.

The results will be recorded using actual application behavior, backend logs, incident records, and before-and-after checks.

2. Safety and Preconditions

All experiments must be performed in a development environment using test data. Database records, email services, and other resources used by real users must not be affected.

Before starting the experiments:

1. Start the SaleSphere frontend, backend, and PatchPilot service.
2. Confirm that the application and database are in a healthy state.
3. Open the System Health and Incidents sections.
4. Keep the browser developer tools and backend terminal visible.
5. Record the initial system status before injecting a failure.
6. Ensure that each experiment can be stopped and the system restored to its normal state.

Readiness Checklist

Component	Expected State
Frontend	SaleSphere dashboard loads correctly
Backend	API and health checks respond
Database	MongoDB connection is available
PatchPilot	Recovery service is running

Test data	Safe demo records are available
Evidence	Logs and incident details can be captured

3. Chaos Injection Mechanism

The chaos testing mechanism allows a tester to deliberately introduce a selected failure instead of waiting for a real problem to occur. Each experiment should use a controlled failure trigger or a dedicated Chaos Control button.

The general testing process is:

1. Select the failure to be tested.
2. Trigger the failure using the configured chaos control.
3. Observe whether SaleSphere detects the problem.
4. Check whether PatchPilot starts the appropriate recovery action.
5. Verify the final application state and record the result.

The failure injection mechanism and the recovery mechanism have separate responsibilities. The chaos control introduces the test failure, while PatchPilot attempts recovery through the configured self-healing workflow.

4. Experiment A — Bad Software Release

Objective: To verify that SaleSphere can detect an unhealthy software release and return to a stable version.

Failure Injection

1. Open the Version Manager section in SaleSphere.
2. Start the controlled bad-release experiment.
3. Observe the update progress and the application's response to the faulty version.
4. Check the incident record and recovery status.

Expected Detection and Recovery

The system should identify that the new version has failed its health check. PatchPilot should initiate the configured rollback process and restore the stable version.

After rollback, the system should perform another health check and update the incident status.

Pass Criteria

The experiment passes if the unhealthy release is detected, the stable version is restored, and the final health check confirms recovery.

Evidence to Capture

Record the faulty version, detection time, rollback start time, recovery time, incident status, and final active version.

5. Experiment B — MongoDB Disconnection

Objective: To test how SaleSphere handles temporary database unavailability without silently losing supported lead operations.

Failure Injection

1. Open the Customer Leads page and note the existing test data.
2. Use a controlled test mechanism to make MongoDB temporarily unavailable to SaleSphere.
3. Attempt a supported lead operation, such as adding a clearly labelled demo lead.
4. Observe the database status, local cache, and pending operation queue.
5. Restore the database connection and inspect the synchronization result.

Expected Detection and Recovery

SaleSphere should detect that the database is unavailable. If offline caching and queuing are working correctly, supported changes should remain pending instead of being discarded.

When the database connection returns, the application should synchronize pending changes and confirm that they have been saved.

Pass Criteria

The experiment passes only if the test operation is retained during the outage, synchronized after reconnection, and saved exactly once in MongoDB.

Evidence to Capture

Record the database status, pending operation count, before-and-after lead data, synchronization logs, and final database record.

6. Experiment C — Email Delivery Failure

Objective: To verify that SaleSphere detects an email failure and executes its configured recovery mechanism.

Failure Injection

1. Open the email failure testing control.
2. Trigger the controlled failure for the next test email.
3. Observe the failure status and PatchPilot's recovery activity.
4. Check the retry result and final email delivery status.

Expected Detection and Recovery

The application should identify the failed email operation and record the relevant status. If retry recovery is implemented, the configured mechanism should attempt delivery again.

The final result must distinguish between a successful retry and an email that remains undelivered.

Pass Criteria

The experiment passes if the failure is detected, the configured recovery action runs, and successful delivery is verified when delivery recovery is the intended outcome.

Evidence to Capture

Record the failure event, retry attempt, recovery status, relevant logs, and delivery confirmation where available.

7. Experiment D — API Failure

Objective: To verify that SaleSphere detects an API failure and reports the system's condition correctly.

Failure Injection

1. Open the System Health section.
2. Trigger a controlled failure response from the selected test API endpoint.
3. Observe the API response, health monitoring status, and incident records.
4. Restore normal API behavior and repeat the health check.

Expected Detection and Recovery

The monitoring mechanism should identify the failed request or unhealthy endpoint. PatchPilot should execute a recovery action only if one is configured for that failure type.

After the test failure is removed, the endpoint should be checked again to confirm whether normal service has returned.

Pass Criteria

The experiment passes if the failure is detected and recorded, the configured recovery behavior is verified, and the final API health check succeeds.

Evidence to Capture

Record the failed endpoint, response status, detection signal, incident details, recovery action, and final health-check result.

8. Recording Results

Complete the following table during testing. Use actual measurements instead of estimated recovery times.

Experiment	Failure Injected	Recovery Verified	Recovery Time	Result
Bad Software Release	Faulty version activated	Stable version restored and verified	_____ sec	Pass / Fail
MongoDB Disconnection	Database made unavailable	Pending changes synchronized and verified	_____ sec	Pass / Fail

Email Failure	Test email failure triggered	Configured retry and delivery result checked	_____ sec	Pass / Fail
API Failure	Controlled API error introduced	Configured recovery and health recheck verified	_____ sec	Pass / Fail

For each experiment, record the following timestamps:

- Failure injected
- Failure detected
- Recovery started
- Recovery verified

Recovery time should be calculated from the failure injection timestamp to the successful verification timestamp. If recovery does not succeed, record the experiment as failed or unresolved rather than reporting a successful recovery time.

9. Rollback and Cleanup

After completing each experiment:

1. Stop the chaos test and remove the injected failure.
2. Restore normal database, API, and email settings.
3. Confirm that SaleSphere and its backend services are healthy.
4. Verify that pending operations are either synchronized or documented as unresolved.
5. Remove or reset demo-only records if required.
6. Save the logs, screenshots, and completed results table.

Only the test environment should be modified during these experiments.

10. Limitations and Honest Reporting

Chaos testing demonstrates only the behavior that has actually been implemented and verified. A failure appearing on the dashboard does not automatically prove that the system has recovered.

Database synchronization must be verified against persisted MongoDB records. Email recovery must be checked against the actual delivery result. API failure detection must be distinguished from automatic service restart.

Some experiments may require additional implementation before they can pass. Any incomplete recovery, data inconsistency, or unhandled failure must be recorded as a limitation.

11. Conclusion

This chaos engineering script provides a repeatable method for testing SaleSphere's resilience against selected failures. By introducing controlled faults, observing PatchPilot's response, and verifying the final system state, the project demonstrates how self-healing behavior can be evaluated using practical evidence.

The final results will depend on the behavior observed during each experiment. The aim is to identify failures early, reduce manual intervention where recovery is supported, and improve the reliability of SaleSphere's CRM operations.